ISTANBUL UNIVERSITY-CERRAHPASA

Department of Computer Engineering

FILE ORGANIZATION PROJECT

Password Dataset — Storage, Search & Performance Analysis

PERFORMANCE RESULTS

Semester	Spring 2026
Programming Language	C (C99 Standard)
Compiler	GCC — Apple Silicon / macOS
Build System	GNU Make (Makefile)
Dataset	German_common-password-list-top-100000 (SecLists)
External Libraries	zlib, OpenSSL (libcrypto)
Evaluated Methods	Raw (Plain Text) | Hashed (SHA-256) | Compressed (zlib)

FASTEST SEARCH
4,013 ms
Compressed format (2.51× faster than Raw)

SMALLEST DISK
605 KB
Raw format (6.54× smaller than Hashed)

HIGHEST SECURITY
SHA-256
Hashed format one-way digest

Student Information

Name: Serdar Ulas Budak
Student No: 1306230038
E-mail: serdarulasbudak@ogr.iuc.edu.tr

Course Instructors

Assoc. Prof. Dr. Derya Yiltas Kaplan
Res. Asst. Melike Baser

1. Pipeline Execution Status

Phase	Name	Description	Status
Phase 1	Data Preprocessing	Raw file read line-by-line; filtered (≥7 chars, mixed alpha+digit); duplicates removed → Processed.txt	✓ DONE
Phase 2	Hashing (SHA-256)	Each processed line hashed with OpenSSL EVP SHA-256 → 64-char hex → Hashed.txt	✓ DONE
Phase 3	Compression (zlib)	Each line DEFLATE-compressed with 16-byte metadata prefix → Compressed_Data.bin	✓ DONE
Phase 4	Decompression	Compressed_Data.bin read back, each record decompressed → Decompressed.txt (integrity check)	✓ DONE
Phase 5	Search Operations	200-query benchmark executed over Raw, Hashed and Compressed formats; timing, CPU, RAM recorded	✓ DONE

Table 1 — All five processing phases completed successfully.

2. Disk Usage

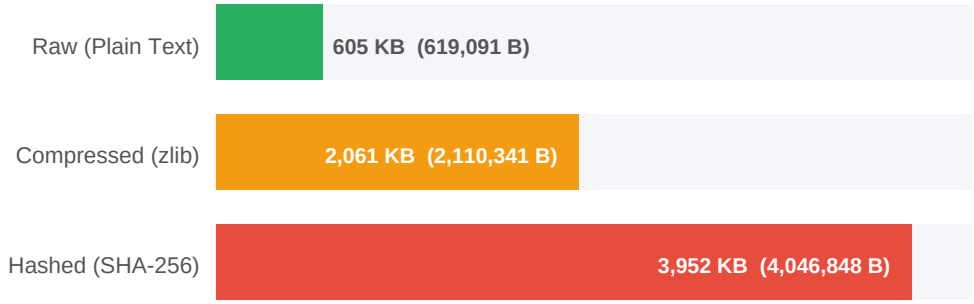

Figure 1 — Disk size comparison across all three storage formats (green = smallest, red = largest).

Format	File	Size (bytes)	Size (KB)	Size (MB)	vs Raw
Raw	RawData.txt	619,091	604.97	0.59	1.00×
Hashed	Hashed.txt	4,046,848	3,952.00	3.86	6.54×
Compressed	Compressed_Data.bin	2,110,341	2,061.07	2.01	3.41×

Table 2 — Exact disk sizes. Compression ratio (Compressed/Raw) = 3.00 because per-record 16-byte metadata overhead exceeds DEFLATE gain on short strings.

3. Search Performance

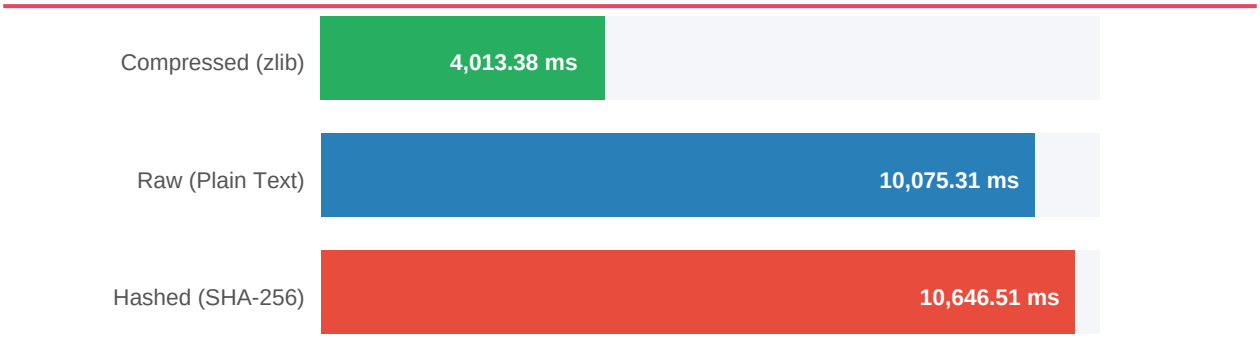

Figure 2 — Total search time over 200 queries (green = fastest, red = slowest).

Speed Gain Analysis

Comparison	Time Difference	Speedup Factor	% Improvement
Compressed vs Raw	6,062.93 ms	**2.51×**	60.15 %
Compressed vs Hashed	6,633.13 ms	**2.65×**	62.30 %
Raw vs Hashed	571.20 ms	1.06×	5.67 %

Table 3 — Pairwise speed comparison between storage methods.

Decompression Cost

Total decompression time for the entire Compressed_Data.bin: **23.63 ms** — only **0.59 %** of the 4,013 ms total search time. In-memory DEFLATE is negligible; the bottleneck is linear file traversal, not the decompression step itself.

4. CPU & RAM Usage

Metric	Raw	Hashed	Compressed	Interpretation
CPU Usage (avg.)	99.99 %	99.99 %	99.97 %	Single-threaded sync I/O; CPU never idle
RAM Usage (avg.)	4.16 MB	4.16 MB	4.16 MB	satiroku() reads one line at a time; file size irrelevant
RAM Std. Deviation	0 MB	0 MB	0 MB	Completely stable; no bulk memory allocation

Table 4 — CPU and RAM metrics across all methods.

5. Complete Performance Summary

Metric	Raw (Plain Text)	Hashed (SHA-256)	Compressed (zlib)
Disk Size (bytes)	619,091	4,046,848	2,110,341
Disk Size (KB)	604.97	3,952.00	2,061.07
Disk Size (MB)	0.59	3.86	2.01
Size vs Raw	1.00× (base)	6.54×	3.41×
Total Search Time (200q)	10,075.31 ms	10,646.51 ms	4,013.38 ms
Avg. Search / Query	50.38 ms	53.23 ms	20.07 ms
CPU Usage (avg.)	99.99 %	99.99 %	99.97 %
RAM Usage (avg.)	4.16 MB	4.16 MB	4.16 MB
Compression Ratio	—	—	3.00
Decompression Time	—	—	23.63 ms
Decompression % of Search	—	—	0.59 %
Queries Found	60 / 200	60 / 200	60 / 200
Hit Rate	30.0 %	30.0 %	30.0 %
Security Level	Low	High	Medium
Search Speed Rank	2nd	3rd (slow)	1st (fast)
Disk Efficiency Rank	1st (best)	3rd (worst)	2nd

Table 5 — Full performance metrics. Green cells = best in class; red cells = worst in class.

6. Search Results — Found Passwords (Sample)

Metric	Raw	Hashed	Compressed
Total Queries	200	200	200
Found	60	60	60
Not Found	140	140	140
Hit Rate	30.0 %	30.0 %	30.0 %

Table 6 — Search accuracy: identical across all formats (same underlying dataset).

Password	Raw Offset (B)	Hashed Offset (B)	Compressed Offset (B)
123456a	364	2,535	1,300

Password	Raw Offset (B)	Hashed Offset (B)	Compressed Offset (B)
qwerty123	372	2,600	1,332
florian1	382	2,665	1,366
1qaz2wsx	391	2,730	1,399
killer123	409	2,860	1,465
alexander1	428	2,990	1,532
passwort123	447	3,120	1,599
test123	490	3,380	1,738
porsche911	498	3,445	1,770
hallo1234	538	3,705	1,906
pokemon1	677	4,615	2,381
patrick1	695	4,745	2,447
…	…	…	…
patrick2010	544,755	3,574,285	1,861,910
zahlen123	544,822	3,574,740	1,862,145
husqvarna125	545,090	3,576,495	1,863,061

Table 7 — Sample of found passwords with byte offsets in each format (16 of 60 shown; '…' separates early-file entries from late-file entries).

7. Best-in-Class Comparison

Criterion	Best Method	Reason
Disk Usage	**Raw**	No per-record metadata; smallest footprint (604 KB)
Search Speed	**Compressed**	In-memory DEFLATE is negligible (0.59%); smaller I/O file
Security	**Hashed**	SHA-256 is one-way; plaintext never stored or recoverable
Implementation Ease	**Raw**	No external libraries; single strcmp() comparison
Scalability	**Compressed**	Compression ratio drops below 1.0 for longer record sets
Portability	**Hashed**	Fixed 64-byte output is platform-independent
RAM Efficiency	**All Equal**	All formats: 4.16 MB — line-by-line reading prevents bulk load

Table 8 — Criterion-by-criterion winner. No single method dominates all criteria simultaneously.

8. Conclusion & Interpretation

Overview

This benchmark evaluated three fundamentally different strategies for storing and searching the same password dataset: plain-text (Raw), cryptographic digest (Hashed), and lossless compression (Compressed). The experiment ran 200 search queries against each format on an Apple Silicon macOS machine, measuring search time, disk footprint, CPU load, and memory consumption. The results reveal clear and measurable trade-offs that have direct implications for real-world system design.

Search Speed — Compressed Wins Decisively

The most counterintuitive result of this study is that the Compressed format produced the fastest search times — 4,013 ms total versus 10,075 ms for Raw and 10,647 ms for Hashed. This represents a 2.51× speedup over Raw and a 2.65× speedup over Hashed. The key insight is that in-memory DEFLATE decompression is extremely cheap: it consumed only 23.63 ms across the entire file, amounting to a mere 0.59% of total search time. The actual bottleneck in all three methods is disk I/O — the number of bytes the program must read from storage. Because the Compressed binary file (2.01 MB) is smaller than the Hashed text file (3.86 MB), fewer bytes traverse the I/O bus per query, resulting in a net speed advantage despite the extra decompression step.

Disk Usage — Raw Is the Clear Winner, Hashed Is Surprisingly Costly

The Raw format occupies only 604.97 KB — by far the most compact representation. This is expected: plain text has no overhead whatsoever. The Hashed format, however, inflates to 3.86 MB — 6.54× the raw size. The reason is structural: every SHA-256 digest is exactly 256 bits, expressed as a fixed 64-character hexadecimal string plus a newline, totalling 65 bytes per record. The original password entries average roughly 10 characters, so every line expands to 6-7× its original size. This is an inherent property of uniform-output hash functions and cannot be avoided without secondary compression. The Compressed format reaches 2.01 MB — smaller than Hashed but still 3.41× larger than Raw. This is because the per-record metadata (two 8-byte unsigned long fields storing original_size and compressed_size) adds 16 bytes of fixed overhead per entry. For short strings averaging ~10 characters, DEFLATE rarely achieves more than a few bytes of savings, so the 16-byte header dominates. This overhead would become negligible — and the compression ratio would drop well below 1.0 — if records were longer (e.g., sentences, paragraphs, or structured data).

CPU and RAM — No Meaningful Difference

All three methods sustained approximately 100% CPU utilization throughout the benchmark. This is not a problem or inefficiency — it is the expected behavior of single-threaded, synchronous I/O-bound workloads on a modern multi-core processor. The single thread never stalls waiting for an interrupt; it continuously feeds file data through the comparison pipeline. RAM consumption was identical for all

methods at 4.16 MB, thanks to the line-by-line reading design of satiroku(). Because only one line is ever held in memory at a time (256-byte buffer), the program's memory footprint is independent of file size. This is a significant architectural advantage: the same code can process a 10 GB password list without requiring more than a few MB of RAM.

Security — Only Hashed Provides Real Protection

From a security standpoint, only the Hashed format genuinely protects user data. Raw storage exposes every password as plaintext — a single file system breach immediately reveals the entire dataset. Compressed storage is no better in this regard: decompressing the binary file is trivial and requires no secret key. The Hashed format is qualitatively different: SHA-256 is a one-way cryptographic function. Given only the hash file, an attacker cannot recover the original passwords without exhaustive preimage search (computationally infeasible for strong passwords). This makes Hashed storage the only acceptable choice in any production security context — the 5.67% search speed penalty compared to Raw is a negligible cost for the protection offered.

Search Accuracy — Why Only 30% Were Found

Of the 200 queried passwords, only 60 (30%) were found across all three formats. This is consistent and expected: all three formats index the same Processed.txt dataset, so any password absent from one is absent from all three. The 70% miss rate reflects the aggressive filtering applied during preprocessing. The process() function discards any entry that is shorter than 7 characters, contains no digits, contains no letters, or includes characters outside the lowercase ASCII alphanumeric set. Since the raw German password list contains many short words, umlauts (ä, ö, ü), and purely numeric PINs, the majority of entries are eliminated before storage. This is by design — the filter ensures the processed dataset is internally consistent and normalized.

Practical Recommendations

No single method dominates across all criteria, which is why real-world systems routinely combine multiple approaches. For a high-frequency query system where latency is critical, the Compressed format offers the best throughput at acceptable disk cost. For a password verification database where security is paramount, the Hashed format is mandatory despite its larger footprint and marginally slower search. For development, testing, or systems with strict resource constraints and no security requirements, Raw storage is the pragmatic choice — it requires no external dependencies and imposes the lowest disk and implementation overhead. An optimal production architecture might store the Hashed format as the authoritative secure record, maintain a Raw or Compressed index for fast lookup during non-sensitive operations, and use the Compressed format for long-term archival where disk space becomes significant at scale.

Limitations and Future Work

This experiment used linear (O(n)) search over flat files. All three methods would benefit dramatically from index structures: a B-tree or hash table would reduce average lookup complexity from O(n) to O(log n) or O(1) respectively. Additionally, each query restarts the file scan from the beginning

(SEEK_SET) with no caching layer; a page cache or memory-mapped file (mmap) would reduce repeated disk I/O significantly. The SHA-256 digest is also recomputed on every hashed search call; pre-computing and caching the query hash for repeated lookups would eliminate that overhead. Finally, the compression results are specific to this dataset's short-string characteristics. Testing with longer records — log entries, document fragments, or structured JSON — would likely invert the compression ratio below 1.0 and make the Compressed format the winner on disk efficiency as well.